An Introduction to the R Programming Language

Part One

Presented by: Samuel Callisto, PhD
Group Leader PKPD, Metrum Research Group
Adjunct Assistant Prof, ECP UMN
25 September 2026

Learner Outcomes

By the end of today's lecture students should be able to

- Explain what the R programming language is
- Name the types of data structures in R
- Be familiar with basic syntax in both base R and tidyverse
- Identify the tidyverse functions associated with different dataset building and cleaning steps, e.g.,
  - `select/distinct`
  - `mutate/summarize`
  - `bind/join`

Base R Syntax

Why is R so hard to learn?

- First and foremost, R is a programming language
  - This is both its greatest strength and largest impediment for new learners
- R is fundamentally different from other statistics tools such as SAS, Stata, SPSS, Excel, etc.
- Many scientists have not taken formal programming courses; YOU ARE NOT ALONE!
- Understanding some basic programming principles will help you become a better R user



Introduction

- The R programming language was created by Ross Ihaka and Robert Gentleman at the University of Auckland in 1993
- R is based on the S programming language, a commercial product initially developed by Bell Labs
- R is free and open source
  - Open source = source code is available for modification and redistribution

How do we interact with R?

- Many users interact with R using the RStudio IDE (integrated development environment)
- Two main flavors of "programs" that you can write in R
    - R scripts
        - Basic script
    - Markdown scripts (Rmd or qmd)
        - Imposes structure on script
        - Produces customizable output documents in various formats (e.g. html, pdf)
- For either type, R will run the commands in the order that they are input, ignoring any line that begins with the comment (#) symbol



Basic Concepts: Working directory

- The location where R will look for files to load and put any files you save

`> setwd("/pmx/classes/ecp8503")` – set working directory

`> getwd()` – get working directory

- Relative path = the filepath from your current working directory
  - `"./data/pk-data.csv"`
- Absolute path = the filepath from the home directory on your computer
  - `"/pmx/classes/ecp8503/data/pk-data.csv"`

Basic Concepts: RStudio Projects

- An RStudio Project keeps all of your R scripts, markdown documents, and data together in one place
- Each project has its own directory, workspace, history, and source documents
  - Different analyses are kept separate from each other
- RStudio: File -> New Project

Creating and using variables

- Use the "assign operator" **<-** to store the value from right hand side in a named variable on the left-hand side
- Unlike some programming languages, in R you do not need to specify the data type for a new variable
- Variables can be overwritten, even as a different data type

```
test <- "hello world"
test
class(test)
test <- 11
test
class(test)
```



Functions

- An object which contains a series of instructions to perform a specific task
  - A function name is always followed by parentheses
  - `log(x)`
  - `c(1, 1, 2, 3, 5)`
- "Arguments" are "passed" into a function to determine the output
  - Some arguments are required, others are optional
  - Some arguments will have a default value
- Base R comes with many functions defined
  - Install packages to load additional functions for specific tasks
  - `help(log)` or `?mean` will open documentation in RStudio
  - Users can easily define their own functions (covered next week)



Data Types

- **Numeric** data are numbers that contain a decimal. **Integers** are whole numbers (those numbers without a decimal point).
- **Logical** (or "boolean") data take on the value of either TRUE or FALSE. There's also another special type of logical called NA to represent missing values.

Data Types, cont.

- **Character** data are used to represent string values. You can think of character strings as something like a word (or multiple words).
- **Factor** data are a special type of character string, with additional attributes (like levels, labels, and an order).

Data Types

- class() function will tell you what type an object is
  - `x <- 12`
  - `class(x)`

    `## [1] "numeric"`

- Logical test to check object class
  - `is.numeric(x)`

    `## [1] TRUE`
  - `is.character(x)`

    `## [1] FALSE`



Typecasting

- Sometimes we may want to convert from one data type to another, this is called “typecasting”
- If R is unable to interpret the data in the requested format, it will return `NA`
  - `NA` is how R represents missing values
  - NAs are “contagious”, meaning that most operations on an NA will return an `NA`

```
as.character(1)
> “1”
as.numeric(“12.0”)
> 12.0
as.numeric(“A”)
> NA
```



Data Structures – Scalars and Vectors

- Scalar – an object containing a single value

```
x <- 12
```

- Vector – an object containing a set of elements, all of the same class

```
y <- 0:12
> y
 [1]  0  1  2  3  4  5  6  7  8  9 10 11 12

names <- c("Bob","Ralph","Mickey")
>names[2]
 [1] "Ralph"
```



Creating numeric vectors

```
1:5
[1] 1 2 3 4 5
seq(from=1, to=5, by=1)
[1] 1 2 3 4 5
rep(1,5)
[1] 1 1 1 1 1
c(4, 2, 3, 1, 5)
[1] 4 2 3 1 5
sort( c(4,2,3,1,5) )
[1] 1 2 3 4 5
```

NOTE: outputs from one function can be used as input to another function using nesting



Creating Factors

- Recall that Factors are an enumerated character data type
- We can think of these as labels corresponding to a numeric value

```
> factor(x=c(1,0,1,1,0), levels = 0:1, labels = c("Female", "Male"))
[1] Male   Female Male   Male   Female
Levels: Female Male
```

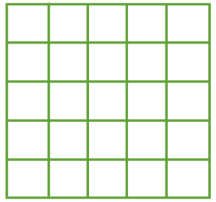


Data Structures – Matrices and Arrays

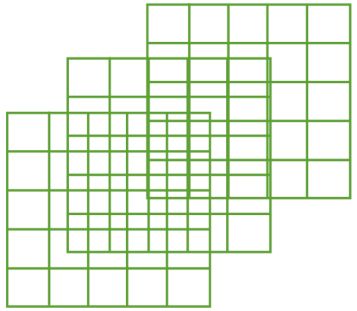
- Matrix – an object containing a square matrix of values, all of the same class

```
> mymat <- matrix(1:16, nrow = 4, byrow = TRUE)
> my_mat
     [,1] [,2] [,3] [,4]
[1,]    1    2    3    4
[2,]    5    6    7    8
[3,]    9   10   11   12
[4,]   13   14   15   16
```

- Array – a multi-dimensional array



matrix



array

Data Structures - Lists

- Lists are objects which contain other objects of various data types.

```
> mylist  <- list(colours = c("black", "yellow", "orange"),
                  evaluation = c(TRUE, TRUE, FALSE, TRUE, FALSE, FALSE),
                  time = matrix(1:6, nrow = 2))

> mylist
$colours
[1] "black"  "yellow" "orange"

$evaluation
[1]  TRUE  TRUE FALSE  TRUE FALSE FALSE

$time
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

MAJOR BENEFIT:
items in the list do not need to be the same length



Data Structures – Data Frames

- A data frame is a two-dimensional object made up of rows and columns
- Each column within the data frame can be of a different type
- All items must be the same length

```
> mydata <- data.frame(COLOR=c("Black","Yellow","Orange"),
                       EVALUATION=c(TRUE,TRUE,FALSE),
                       VALUE=c(1,3,6))
> mydata
   COLOR EVALUATION VALUE
1  Black       TRUE     1
2 Yellow       TRUE     3
3 Orange      FALSE     6
```



Data Structures – Data Frames

- Dimension of data frame

```
> dim(mydata)
[1] 3 3
```

- Accessing individual columns

```
> mydata$COLOR
[1] Black  Yellow Orange
Levels: Black Orange Yellow
> mydata[["VALUE"]]
[1] 1 3 6
```

- Accessing individual rows

```
> mydata[2,]
   COLOR EVALUATION VALUE
2 Yellow       TRUE     3
```



Some useful mathematical operations

`+ - * /`	arithmetic operations
`x^2`	x squared
`sqrt(x)`	square root of x
`y %% x`	remainder of y/x (modulo operator)
`round(x, digits=3)`	rounds x to the nearest thousandth
`floor(x)`	round down to nearest integer
`ceiling(x)`	round up to nearest integer
`sum(x)`	sum of all values in the vector x
`quantile(x, 0.9)`	90th percentile of the vector x
`mean(x)`	mean of the vector x

Most functions will operate on either scalar or every item in a vector with identical syntax



Read and Write Data

base R functions also exist (read.csv), and other data structures are available

- The readr package makes it easy to read or write multiple file formats using functions that start with read_* or write_*

Functions		Description
read_csv() or read_csv2()	write_csv()	comma or semicolon (CSV)
read_delim()	write_delim()	general separator
read_table()	write_table()	whitespace-separated

- The imported tables are of class tibble *(tbl_df)*, a modern version of data.frame from the tibble package

```
> library(readr)
> nmdata <- read_csv("pk-nonmem-example.csv")
> dim(nmdata)
[1] 1488   27
```

argument is a relative path from your working directory



Mike's Unoffical Style Guide

- Avoid using more than 80 characters per line to allow reading the complete code
- Always use a space after a comma, never before
- The operators (==, +, -, <-, %>%, etc.) should have a space before and after
- There is no space between the name of a function and the first parenthesis, nor between the last argument and the final parenthesis of a function
- Avoid reusing names of functions and common variables (`c <- 5` masks `c()`)
- Sort the script separating the parts with the comment form
- `# Import data -----`
- Avoid accent marks or special symbols in names, files, routes, etc.
- Object names should follow a constant structure (`day_one` vs. `day_1`)



tidyverse

tidyverse

- The tidyverse is a collection of packages specially focused on data science
- The core of the tidyverse collection is made up of the following packages:

tibble	Modern and effective table system
dplyr	Grammar for data manipulation
tidyr	Set of functions to create tidy data
stringr	Function set to work with characters
readr	An easy and fast way to import data
forcats	Tools to easily work with factors

- To install: `install.packages("tidyverse")`
- To load: `library(tidyverse)`



What's a tibble?

- A tibble is a special type of data.frame used by tidyverse
- Includes and displays metadata about each column
- data.frames are automatically converted to tibbles as outputs of tidyverse functions
- For all intents and purposes, tibbles and data.frames can be treated the same
- Can manually convert back and forth using typecasting functions `as_tibble()` or `as.data.frame()`



Pipe %>%

- The *pipe* operator, %>%, allows the user to combine functions without the need to assign the result to a new object
- The *pipe* operator passes the output of a function to the first argument of the next function
- This way of combining functions allows you to chain together sequential tasks

- Trivial example:

```
1:5 %>% mean()
[1] 3
```

There is also a new pipe operator |> in base R that mostly works the same



major data manipulation verbs

Verb	Usage
filter	keep matching row criteria
summarize	calculates summaries, returns reduced output
mutate	add new variables to existing data.frame
select	select columns by name
arrange	reorder rows

```
df      <-      data.frame(
        ID      = 1:5,
        GENDER =        c("MALE", "MALE", "FEMALE",
                                  "MALE", "FEMALE"),
        WT      =       c(70, 76, 60, 64, 68))
```

ID	GENDER	WT
1	MALE	70
2	MALE	76
3	FEMALE	60
4	MALE	64
5	FEMALE	68



Verb	Usage
filter	keep matching row criteria
summarize	calculates summaries, returns reduced output
mutate	add new variables to existing data.frame
select	select columns by name
arrange	reorder rows

```
filter(df, GENDER == "FEMALE")
```

ID	GENDER	WT
1	MALE	70
2	MALE	76
3	FEMALE	60
4	MALE	64
5	FEMALE	68



ID	GENDER	WT
3	FEMALE	60
5	FEMALE	68

common dplyr filter (subset) operators

operator	meaning
==, !=	equal, not equal
>, >=	greater than, greater than or equal to
<, <=	less than, less than or equal to
is.na(), !is.na()	is NA, not NA
!duplicated()	only first value
%in%	in specified values

filter separator	base equivalent	meaning
,	&	and
|	|	or



```
filter(df, ID %in% c(1, 3, 5))
```

ID	GENDER	WT
1	MALE	70
2	MALE	76
3	FEMALE	60
4	MALE	64
5	FEMALE	68



ID	GENDER	WT
1	MALE	70
3	FEMALE	60
5	FEMALE	68

```
filter(df, GENDER == "MALE", WT > 70)
```

ID	GENDER	WT
1	MALE	70
2	MALE	76
3	FEMALE	60
4	MALE	64
5	FEMALE	68



ID	GENDER	WT
2	MALE	76

```
filter(df, GENDER == "FEMALE" | WT < 70)
```

ID	GENDER	WT
1	MALE	70
2	MALE	76
3	FEMALE	60
4	MALE	64
5	FEMALE	68



ID	GENDER	WT
3	FEMALE	60
4	MALE	64
5	FEMALE	68

Verb	Usage
filter	keep matching row criteria
summarize	calculates summaries, returns reduced output
mutate	add new variables to existing data.frame
select	select columns by name
arrange	reorder rows

```
df %>%
summarize(meanWT = mean(WT))
```

ID	GENDER	WT
1	MALE	70
2	MALE	76
3	FEMALE	60
4	MALE	64
5	FEMALE	68



meanWT
67.6

```
summarize(df, meanWT = mean(WT))
```

```
df %>% group_by(GENDER) %>%
    summarize(meanWT = mean(WT)) %>%
    ungroup()
```

ID	GENDER	WT
1	MALE	70
2	MALE	76
3	FEMALE	60
4	MALE	64
5	FEMALE	68



GENDER	meanWT
MALE	70
FEMALE	64

You should always ungroup after grouping, but I will omit it from here on out to save space

```
df %>% group_by(GENDER) %>%
    summarize(meanWT = mean(WT),
              n = n())
```

ID	GENDER	WT
1	MALE	70
2	MALE	76
3	FEMALE	60
4	MALE	64
5	FEMALE	68



GENDER	meanWT	n
MALE	70	3
FEMALE	64	2

Verb	Usage
filter	keep matching row criteria
summarize	calculates summaries, returns reduced output
mutate	add new variables to existing data.frame
select	select columns by name
arrange	reorder rows

```
df %>% mutate(
    meanWT = mean(WT))
```

ID	GENDER	WT
1	MALE	70
2	MALE	76
3	FEMALE	60
4	MALE	64
5	FEMALE	68



ID	GENDER	WT	meanWT
1	MALE	70	67.6
2	MALE	76	67.6
3	FEMALE	60	67.6
4	MALE	64	67.6
5	FEMALE	68	67.6

```
df %>%
  group_by(GENDER) %>%
  mutate(meanWT = mean(WT))
```

ID	GENDER	WT
1	MALE	70
2	MALE	76
3	FEMALE	60
4	MALE	64
5	FEMALE	68



ID	GENDER	WT	meanWT
1	MALE	70	70
2	MALE	76	70
3	FEMALE	60	64
4	MALE	64	70
5	FEMALE	68	64

```
df %>% group_by(GENDER) %>%
 mutate(meanWT = mean(WT),
        mWT_LB = meanWT*2.2)
```

ID	GENDER	WT
1	MALE	70
2	MALE	76
3	FEMALE	60
4	MALE	64
5	FEMALE	68



ID	GENDER	WT	meanWT	mWT_LB
1	MALE	70	70	154
2	MALE	76	70	154
3	FEMALE	60	64	140.8
4	MALE	64	70	154
5	FEMALE	68	64	140.8

```
df %>%
  mutate(ISM = if_else(GENDER == "MALE", 1, 0))
```

ID	GENDER	WT
1	MALE	70
2	MALE	76
3	FEMALE	60
4	MALE	64
5	FEMALE	68





A brief aside on conditional statements

- R has several ways to conditionally apply a function or return a value based on some conditional statement
- The most commonly used function in data analysis is if_else()
  - switch statements are implemented using case_when() function (see backups)

conditional statement

returned if FALSE

```
if_else(GENDER == "MALE", 1, 0)
```

function call

returned if TRUE

```
df %>%
  mutate(ISM = if_else(GENDER == "MALE", 1, 0))
```

ID	GENDER	WT
1	MALE	70
2	MALE	76
3	FEMALE	60
4	MALE	64
5	FEMALE	68



ID	GENDER	WT	ISM
1	MALE	70	1
2	MALE	76	1
3	FEMALE	60	0
4	MALE	64	1
5	FEMALE	68	0

Verb	Usage
filter	keep matching row criteria
summarize	calculates summaries, returns reduced output
mutate	add new variables to existing data.frame
select	select columns by name
arrange	reorder rows

```
df2 %>% select(ID, WT)
```

ID	GENDER	WT	meanWT
1	MALE	70	67.6
2	MALE	76	67.6
3	FEMALE	60	67.6
4	MALE	64	67.6
5	FEMALE	68	67.6



ID	WT
1	70
2	76
3	60
4	64
5	68

```
df2 %>% select(GENDER:meanWT)
```

ID	GENDER	WT	meanWT
1	MALE	70	67.6
2	MALE	76	67.6
3	FEMALE	60	67.6
4	MALE	64	67.6
5	FEMALE	68	67.6



GENDER	WT	meanWT
MALE	70	67.6
MALE	76	67.6
FEMALE	60	67.6
MALE	64	67.6
FEMALE	68	67.6

```
df2 %>% select(-ID)
```

ID	GENDER	WT	meanWT
1	MALE	70	67.6
2	MALE	76	67.6
3	FEMALE	60	67.6
4	MALE	64	67.6
5	FEMALE	68	67.6



GENDER	WT	meanWT
MALE	70	67.6
MALE	76	67.6
FEMALE	60	67.6
MALE	64	67.6
FEMALE	68	67.6

```
df2 %>% select(ID, WEIGHT = WT)
```

ID	GENDER	WT	meanWT
1	MALE	70	67.6
2	MALE	76	67.6
3	FEMALE	60	67.6
4	MALE	64	67.6
5	FEMALE	68	67.6



ID	WEIGHT
1	70
2	76
3	60
4	64
5	68

```
df2 %>% rename(WEIGHT = WT)
```

ID	GENDER	WT	meanWT
1	MALE	70	67.6
2	MALE	76	67.6
3	FEMALE	60	67.6
4	MALE	64	67.6
5	FEMALE	68	67.6



ID	GENDER	WEIGHT	meanWT
1	MALE	70	67.6
2	MALE	76	67.6
3	FEMALE	60	67.6
4	MALE	64	67.6
5	FEMALE	68	67.6

```
df %>% select(<function>(<arguments>))
```

df with the following columns:

WEIGHT	WEIGHT_KG	MEAN_WEIGHT	OCC1	OCC2	OCC3	OCC4	HEIGHT

function	meaning	example	columns selected
`starts_with`	names start with	`starts_with("WEIGHT"))`	`WEIGHT, WEIGHT_KG`
`ends_with`	names ends with	`ends_with("GHT")`	`WEIGHT, MEAN_WEIGHT, HEIGHT`
`contains`	names contains	`contains("EI")`	`WEIGHT, WEIGHT_KG, MEAN_WEIGHT, HEIGHT`
`matches`	regular expression matching	`matches("_")`	`WEIGHT_KG, MEAN_WEIGHT`
`num_range`	specify range of columns with consistent names with numeric suffix	`num_range("OCC",1:3)`	`OCC1, OCC2, OCC3`



Verb	Usage
filter	keep matching row criteria
summarize	calculates summaries, returns reduced output
mutate	add new variables to existing data.frame
select	select columns by name
arrange	reorder rows

```
df %>% arrange(WT)
```

ID	GENDER	WT
1	MALE	70
2	MALE	76
3	FEMALE	60
4	MALE	64
5	FEMALE	68



ID	GENDER	WT
3	FEMALE	60
4	MALE	64
5	FEMALE	68
1	MALE	70
2	MALE	76

lowest weight



highest weight

```
df %>% arrange(desc(WT))
```

ID	GENDER	WT
1	MALE	70
2	MALE	76
3	FEMALE	60
4	MALE	64
5	FEMALE	68



ID	GENDER	WT
2	MALE	76
1	MALE	70
5	FEMALE	68
4	MALE	64
3	FEMALE	60

highest weight



lowest weight

Learn from my mistakes...

```
> arrange(5:1)
Error in UseMethod("arrange") :
  no applicable method for 'arrange' applied to an
object of class "c('integer', 'numeric')"
```

```
> df %>% sort(WEIGHT)
Error: object 'WEIGHT' not found
```

```
> sort(5:1)
[1] 1 2 3 4 5
```

`sort` works on vectors, `arrange` works on columns in data.frames



minor data manipulation verbs

Verb	Usage
distinct	keep all distinct elements per key
slice	return certain rows by number (overall or within group)
rename	rename columns
relocate	move a column within the data frame

```
head(Theoph)
```

Subject	Wt	Dose	Time	conc
1	79.6	4.02	0	0.74
1	79.6	4.02	0.25	2.84
1	79.6	4.02	0.57	6.57
1	79.6	4.02	1.12	10.5
1	79.6	4.02	2.02	9.66
1	79.6	4.02	3.82	8.58

```
Theoph %>%
distinct(Subject)
```

Subject
3
6
7
8
11



```
head(Theoph)
```

Subject	Wt	Dose	Time	conc
1	79.6	4.02	0	0.74
1	79.6	4.02	0.25	2.84
1	79.6	4.02	0.57	6.57
1	79.6	4.02	1.12	10.5
1	79.6	4.02	2.02	9.66
1	79.6	4.02	3.82	8.58

```
Theoph %>%
distinct(Subject,
         .keep_all = TRUE)
```

Subject	Wt	Dose	Time	conc
1	79.6	4.02	0	0.74
2	72.4	4.4	0	0
3	70.5	4.53	0	0
4	72.7	4.4	0	0
5	54.6	5.86	0	0



```
Theoph %>%
group_by(Subject) %>%
slice(1:2)
```

Subject	Wt	Dose	Time	conc
6	80.0	4.00	0.00	0.00
6	80.0	4.00	0.27	1.29
7	64.6	4.95	0.00	0.15
7	64.6	4.95	0.25	0.85
8	70.5	4.53	0.00	0.00
8	70.5	4.53	0.25	3.05
11	65.0	4.92	0.00	0.00
11	65.0	4.92	0.25	4.86
3	70.5	4.53	0.00	0.00
3	70.5	4.53	0.27	4.40

```
Theoph %>%
group_by(Subject) %>%
slice(c(1, n()))
```

Subject	Wt	Dose	Time	conc
6	80.0	4.00	0.00	0.00
6	80.0	4.00	23.85	0.92
7	64.6	4.95	0.00	0.15
7	64.6	4.95	24.22	1.15
8	70.5	4.53	0.00	0.00
8	70.5	4.53	24.12	1.25
11	65.0	4.92	0.00	0.00
11	65.0	4.92	24.08	0.86
3	70.5	4.53	0.00	0.00
3	70.5	4.53	24.17	1.05



```
Theoph %>%
   rename(ID = Subject)
```

ID	Wt	Dose	Time	conc
3	70.5	4.53	0.00	0.00
3	70.5	4.53	0.27	4.40

```
head(Theoph)
```

Subject	Wt	Dose	Time	conc
1	79.6	4.02	0	0.74
1	79.6	4.02	0.25	2.84
1	79.6	4.02	0.57	6.57
1	79.6	4.02	1.12	10.5
1	79.6	4.02	2.02	9.66
1	79.6	4.02	3.82	8.58

```
Theoph %>%
  relocate(Wt, .after=conc)
```

Subject	Dose	Time	conc	Wt
1	4.02	0	0.74	79.6
1	4.02	0.25	2.84	79.6
1	4.02	0.57	6.57	79.6
1	4.02	1.12	10.5	79.6
1	4.02	2.02	9.66	79.6
1	4.02	3.82	8.58	79.6



dplyr verbs

```
bind_rows(x_df, y_df)
```

```
dosing_df <- data.frame(ID = 1:2, TIME =0,
                        AMT = 100, MDV = 1)
sample_df <- data.frame(expand.grid(ID = 1:2, TIME = seq(0, 2, 1),
                                    AMT = 0, MDV = 0))

df3 <- bind_rows(sample_df,
                 dosing_df)
```

ID	TIME	AMT	MDV
1	0	0	0
2	0	0	0
1	1	0	0
2	1	0	0
1	2	0	0
2	2	0	0
1	0	100	1
2	0	100	1

* expand.grid is a very handy function for generating permutations
* MDV = missing dependent variable (a NONMEM-style flag column)



```
df3 %>% arrange(ID, TIME, desc(MDV))
```

ID	TIME	AMT	MDV
1	0	0	0
2	0	0	0
1	1	0	0
2	1	0	0
1	2	0	0
2	2	0	0
1	0	100	1
2	0	100	1



ID	TIME	AMT	MDV
1	0	100	1
1	0	0	0
1	1	0	0
1	2	0	0
2	0	100	1
2	0	0	0
2	1	0	0
2	2	0	0

```
<join>(x_df, y_df)
```

```
idtime <- data.frame(
  expand.grid(
      ID = as.numeric(1:3),
      TIME = c(0,1)
  )
) %>% arrange(ID)
idwt <- data.frame(
      ID = c(1, 2, 4), WT = c(70, 80, 75))
```

ID	TIME
1	0
1	1
2	0
2	1
3	0
3	1

idtime

ID	WT
1	70
2	80
4	75

idwt



joins

- The _join() functions add columns from one data frame to another, matching rows based on the values in common columns.

Function	Description
inner_join(x,y)	Includes all rows that occur in both x and y
left_join(x,y)	Includes all rows that occur in x
right_join(x,y)	Includes all rows that occur in y
full_join(x,y)	Includes all rows that occur in either x or y



INNER JOIN

idtime/idwt => in both

ID	TIME
1	0
1	1
2	0
2	1
3	0
3	1

idtime

ID	WT
1	70
2	80
4	75

idwt

inner_join(idtime, idwt)

ID	TIME	WT
1	0	70
1	1	70
2	0	80
2	1	80

ID	WT	TIME
1	70	0
1	70	1
2	80	0
2	80	1

inner_join(idwt, idtime)



LEFT JOIN

idtime/idwt => in both

ID	TIME
1	0
1	1
2	0
2	1
3	0
3	1

idtime

ID	WT
1	70
2	80
4	75

idwt

left_join(idtime, idwt)

ID	TIME	WT
1	0	70
1	1	70
2	0	80
2	1	80
3	0	NA
3	1	NA

left_join(idwt, idtime)

ID	WT	TIME
1	70	0
1	70	1
2	80	0
2	80	1
4	75	NA



FULL JOIN

idtime/idwt => in both

ID	TIME
1	0
1	1
2	0
2	1
3	0
3	1

idtime

ID	WT
1	70
2	80
4	75

idwt

full_join(idtime, idwt)

ID	TIME	WT
1	0	70
1	1	70
2	0	80
2	1	80
3	0	NA
3	1	NA
4	NA	75

ID	WT	TIME
1	70	0
1	70	1
2	80	0
2	80	1
3	NA	0
3	NA	1
4	75	NA

full_join(idwt, idtime)



Join	Usage
semi_join	return all rows from x where there are matching values in y, keeping just columns from x.
anti_join	return all rows from x where there are not matching values in y, keeping just columns from x.
cross_join	returns all combinations of rows from x and y (similar to `expand.grid()`, but takes data.frames instead of vectors)



SEMI JOIN

idtime/idwt => in both

ID	TIME
1	0
1	1
2	0
2	1
3	0
3	1

idtime

ID	WT
1	70
2	80
4	75

idwt

semi_join(idtime, idwt)

ID	TIME
1	0
1	1
2	0
2	1

ID	WT
1	70
2	80

semi_join(idwt, idtime)

* A semi join differs from an inner join because an inner join will return one row of x for each matching row of y, where a semi join will never duplicate rows of x

ANTI JOIN

idtime/idwt => in both

ID	TIME
1	0
1	1
2	0
2	1
3	0
3	1

idtime

ID	WT
1	70
2	80
4	75

idwt

anti_join(idtime, idwt)

ID	TIME
3	0
3	1

ID	WT
4	75

anti_join(idwt, idtime)



CROSS JOIN

```
df3 <- bind_rows(
    cross_join(data.frame(ID = 1:2, AMT = 0, MDV = 0), data.frame(TIME = 0:2)),
    data.frame(ID = 1:2, TIME = 0, AMT = 100, MDV = 1)
) %>% arrange(ID, TIME, desc(MDV))
```

ID	TIME	AMT	MDV
1	0	100	1
1	0	0	0
1	1	0	0
1	2	0	0
2	0	100	1
2	0	0	0
2	1	0	0
2	2	0	0

* Compare syntax to slide 70



```
pivot_<direction>(df)
```

pivots

- The pivot_longer() function “pivots” the data to make it longer, with more rows and fewer columns
  - The values from multiple columns are placed in one column, with another column containing the previous variable names
  - Often useful for ”stacking” data prior to plotting, when you want to colorize plots by column
- The pivot_wider() function “pivots” the data to make it wider, with fewer rows and more columns
  - Pulls values from one column and the associated names from another to create additional columns
- replaces “gather” and “spread” syntax as of tidyr 1.0.0



PIVOT WIDER

ID	name	value
1	SEX	M
1	AGE	45
1	WT	77.2
4	SEX	M
4	AGE	36
4	WT	72.3
7	SEX	F
7	AGE	64
7	WT	119

Useful for converting from SDTM/ADaM-style data set to a NONMEM-style data set

```
df %>%
pivot_wider(id_cols=ID,
names_from = name,
values_from = value)
```

ID	SEX	AGE	WT
1	M	45	77.2
4	M	36	72.3
7	F	64	119



PIVOT LONGER

ID	SEX	AGE	WT
1	M	45	77.2
4	M	36	72.3
7	F	64	119

```
df %>%
pivot_longer(cols=SEX:WT,
    names_to = "name",
   values_to = "value")
```

ID	name	value
1	SEX	M
1	AGE	45
1	WT	77.2
4	SEX	M
4	AGE	36
4	WT	72.3
7	SEX	F
7	AGE	64
7	WT	119



Character Manipulation

- For working with strings we can use the stringr package, whose functions always start with str_* followed by a verb and the first argument

Function	Description
str_replace()	replace patterns
str_c()	combine characters
str_detect()	detect patterns
str_extract()	extract patterns
str_sub()	extract by position
str_length()	length of string

Additional Resources

https://intro2r.com/ - Introduction to Base R and RStudio

https://dominicroye.github.io/en/2020/a-very-short-introduction-to-tidyverse/
A short introduction to the Tidyverse

https://www.rstudio.com/resources/cheatsheets/
Quick reference guides for tidyverse functions

https://r4ds.had.co.nz/
Free textbook "R for Data Science" by Hadley Wickham covering many topics

